

Cognome.....Nome.....

Matricola.....Turno.....

Algoritmi e Strutture Dati

Appello dell'11-07-2012 – Dott. Leporati / Prof. Zandron

– Scritto completo –

Esercizio 1

Scrivere un algoritmo che risolva il seguente problema, utilizzando la tecnica **divide-et-impera**: dato in input un vettore $V[1..n]$ contenente $n > 0$ valori interi, calcolare e restituire il valore:

$$(V[1] * V[2]) + (V[2] * V[3]) + (V[3] * V[4]) + \dots + (V[n-1] * V[n])$$

Scrivere poi l'**equazione di ricorrenza** che esprime il tempo di calcolo di tale algoritmo, e **risolverla** utilizzando un metodo a piacere.

Esercizio 2

Scrivere un algoritmo **iterativo** che, dati in ingresso due vettori $A[1..m]$ e $B[1..n]$ contenenti rispettivamente m ed n caratteri dell'alfabeto italiano, conti quante volte la stringa (sequenza di caratteri) memorizzata in A compare in B . Si assuma che **n sia molto maggiore di m** .

Valutare poi i **tempi di calcolo** nei casi *migliore* e *peggiore*, indicando quante volte viene eseguita ogni operazione in pseudocodice. Si assuma che ogni singola operazione elementare impieghi lo stesso tempo (costante).

Esempio: dati i vettori:

$A = [a, b, a]$

$B = [c, d, c, a, b, a, b, a, c, a, b, a, b, c, d, b, a, b, b, a, b, a, b]$

il programma deve stampare il valore 4, dato che la stringa "aba" compare 4 volte in B (a partire dalle posizioni 4, 6, 10 e 20. **Fare attenzione alle eventuali sovrapposizioni!**).

Esercizio 3

Simulare l'esecuzione dell'algoritmo HEAPSORT sul seguente array:

$A = [70, 100, 30, 20, 90, 80]$

illustrando i vari passaggi, e mostrando lo stato dell'array **dopo ogni operazione di scambio** fra i suoi elementi. In particolare, si mostrino anzitutto gli scambi effettuati dalla funzione BUILDHEAP per risistemare gli elementi del vettore in modo da formare uno heap, e successivamente si mostrino gli scambi effettuati dall'algoritmo di ordinamento vero e proprio per ordinare gli elementi in ordine crescente. Se lo si ritiene opportuno, l'array può essere rappresentato in forma di albero; in tal caso, sul primo albero si indichino in maniera chiara gli indici degli elementi.

Esercizio 4

Costruire un **albero binario di ricerca** inserendo *successivamente* (uno dopo l'altro) i seguenti valori:

100, 41, 10, 223, 134, 5, 43, 42, 90, 192, 199

Disegnare gli alberi ottenuti **dopo ogni inserimento**.

Si mostrino poi:

- le sequenze di interi ottenute visitando l'albero finale negli ordini **anticipato**, **simmetrico** e **posticipato** (ovvero, rispettivamente: pre-ordine, in-ordine e post-ordine);
- gli alberi ottenuti cancellando prima la chiave 90, poi la chiave 134 e infine la chiave 41 (mostrare la struttura dell'albero ottenuta dopo *ogni singola cancellazione, a partire dall'albero precedente*). Spiegare brevemente come vengono fatte le operazioni di cancellazione.